

Take Home Assignment

Senior Full Stack Engineer, Cross Collaborative AI Platform

Context

We are hiring the engineer who will help build our internal AI platform from MVP to production. This exercise mirrors the actual work. You will design and ship a small but real multi-agent AI application, deploy it, and walk us through your decisions.

We care far more about how you think and what you choose to build than about completeness. A focused, well-reasoned app beats a sprawling, half working one. Treat this like a real production task with a tight deadline, because that is the job.

Time box: Please spend no more than 8 to 10 hours. We are not looking for a finished product. We are looking for senior judgement under constraint. If you run out of time, stop and tell us what you would do next in your notes.

The Problem

Build a multi-agent assistant that plans trips. A user describes what they want in plain language, for example a five day trip somewhere warm in Europe for under 1500 pounds, and the system routes the request to, or orchestrates across, three specialised AI agents. Each agent has its own knowledge focus, tools, and behavioural rules.

Agent	Role	Behavioural constraint
Destination Agent	Suggests destinations that fit the user's stated preferences, such as climate, interests, and budget band.	Must justify each suggestion against the stated preferences, and must never recommend a destination that breaks a hard constraint the user gave.
Itinerary Agent	Builds a day by day plan for a chosen destination and trip length. May consult the Destination Agent for context.	Each day must be realistic on travel time and sequencing, and it must say so when it is uncertain.
Budget Agent	Estimates the total cost of a plan and checks it against the user's budget.	It must never silently exceed the budget. It flags the overage and proposes a cheaper alternative.

The interesting part, and the core of this exercise, is the orchestration layer. You decide which agent or agents a query needs, chain them when one depends on another (for example Destination, then Itinerary, then Budget), and synthesise a single coherent answer. The agents rely on the language model's general knowledge, so no dataset is provided or required. If you want to call a free external API, that is welcome but not expected.

What we want you to build

Must have, the core

- A working web app where a user submits a natural language request and receives a synthesised answer.
- Three distinct agents with separate knowledge focus and behaviour, plus an orchestration layer that routes and, where needed, chains them.
- The answer should show which agent or agents contributed, because transparency matters in a real tool.
- A **React 18 and TypeScript** frontend built with Vite. Backend in Node.js or Python, your choice.

- Some persistence. At a minimum, a log of requests and which agents handled each one, which mirrors our real requirement for an audit trail of AI interactions.
- Deployed and live, reachable by a URL.

Should have, if time allows

- Streaming responses, or a clear loading and agent activity state.
- Sensible error handling when an agent or the model call fails.
- A thoughtful data schema, since you will be designing persistence and audit schemas in the real role.

Stretch, genuinely optional, note it rather than rush it

- A second user role that sees different behaviour or data. We go deep on authentication and access control in the live round, so a simple stub is fine. Please do not build enterprise identity for this.
- Light observability, such as a small metrics or logs view.

Please do not spend your hours on visual polish, full authentication, or feature breadth. We would rather see three agents orchestrated well than six built shallowly.

Cost and tooling, you should pay nothing

- **Language model:** Use any free tier, such as Google AI Studio (Gemini), Groq, or OpenRouter's free models. Do not pay for an API key. Commit a `.env.example` and never your real key.
- **Hosting:** Deploy to any free tier, such as Vercel, Netlify, Render, Supabase, or Fly.io. No paid hosting is required. We just need a live URL.
- Using AI coding assistants such as Copilot, Cursor, or Claude is fine and expected, but you must fully understand and be able to defend every line, because you will walk us through it.

What to submit

1. **A live URL** to the deployed app.
2. **A GitHub repository**, public or shared with us, including a short README on how to run it.
3. **A decision note** (max 400 words, or a 10 minute Loom) covering your three biggest design decisions and why, especially the orchestration approach, and what you deliberately cut or simplified, and why.
4. **A production architecture note** (max 300 words). Describe how you would deploy and scale this application on Azure to support 500 or more concurrent users. Cover how you would provision and manage the infrastructure, how you would handle enterprise authentication and access control, and how you would monitor the platform in production. We are interested in the choices you make and the reasons behind them.

What happens next

If your submission moves forward, the next stage is a 60 minute live session. You will screen share and walk us through your code and decisions, and we will explore how you would scale it, including concurrency, enterprise authentication, and hardening the agent layer for production. Build something you are ready to defend.